복합 인덱스 성능 테스트

복합 인덱스 성능 테스트 결과

프로젝트: LapPick
목적: (GOODS_NUM, IPGO_QTY) 복합 인덱스의 Covering Index 효과 검증

테스트 결과 요약

항목	케이스 A (인덱스 없음)	케이스 B (복합 인덱스)	개선
실행 방식	TABLE ACCESS FULL	INDEX RANGE SCAN	Covering Index
옵티마이저 Cost	1,654	22	98.7% 감소
평균 응답 시간	167ms	0.3ms	약 557배 향상
테이블 접근	있음	없음	완전 제거
Predicate 방식	filter()	access()	-

---

1. 테스트 환경

항목	내용
DBMS	Oracle Database XE 21c
테스트 테이블	TEST_GOODS_IPGO (GOODS_IPGO 구조 동일)
총 데이터	1,000,000건 (상품 200개 × 5,000건 수불부 이력)
테스트 쿼리	SELECT SUM(IPGO_QTY) FROM TEST_GOODS_IPGO WHERE GOODS_NUM = ?
측정 도구	DBMS_UTILITY.GET_TIME (정밀도 10ms)
측정 방식	100회 반복 후 총 시간 / 100 = 회당 평균 (정밀도 보완)

> **측정 방식 보충**: DBMS_UTILITY.GET_TIME의 정밀도는 10ms로, 인덱스 적용 후 빠른 쿼리는 단건 측정으로 잡히지 않는다. 100회 반복 실행 후 총 시간을 나누는 방식은 DB 성능 측정의 표준 벤치마킹 기법이다.

---

2. 테스트 케이스

케이스 A: 복합 인덱스 없음

```
-- PK 인덱스(IPGO_NUM)만 존재
-- GOODS_NUM 인덱스 없음
SELECT SUM(IPGO_QTY) AS STOCK_QTY
FROM TEST_GOODS_IPGO
WHERE GOODS_NUM = 'goods_000001';
```

케이스 B: 복합 인덱스 적용 (Covering Index)

```
-- 인덱스 구조: (GOODS_NUM, IPGO_QTY)
-- WHERE 조건 컬럼 + SUM 집계 대상 컬럼을 모두 인덱스에 포함
CREATE INDEX IDX_TEST_IPGO_NUM_QTY
    ON TEST_GOODS_IPGO (GOODS_NUM, IPGO_QTY);
```

---

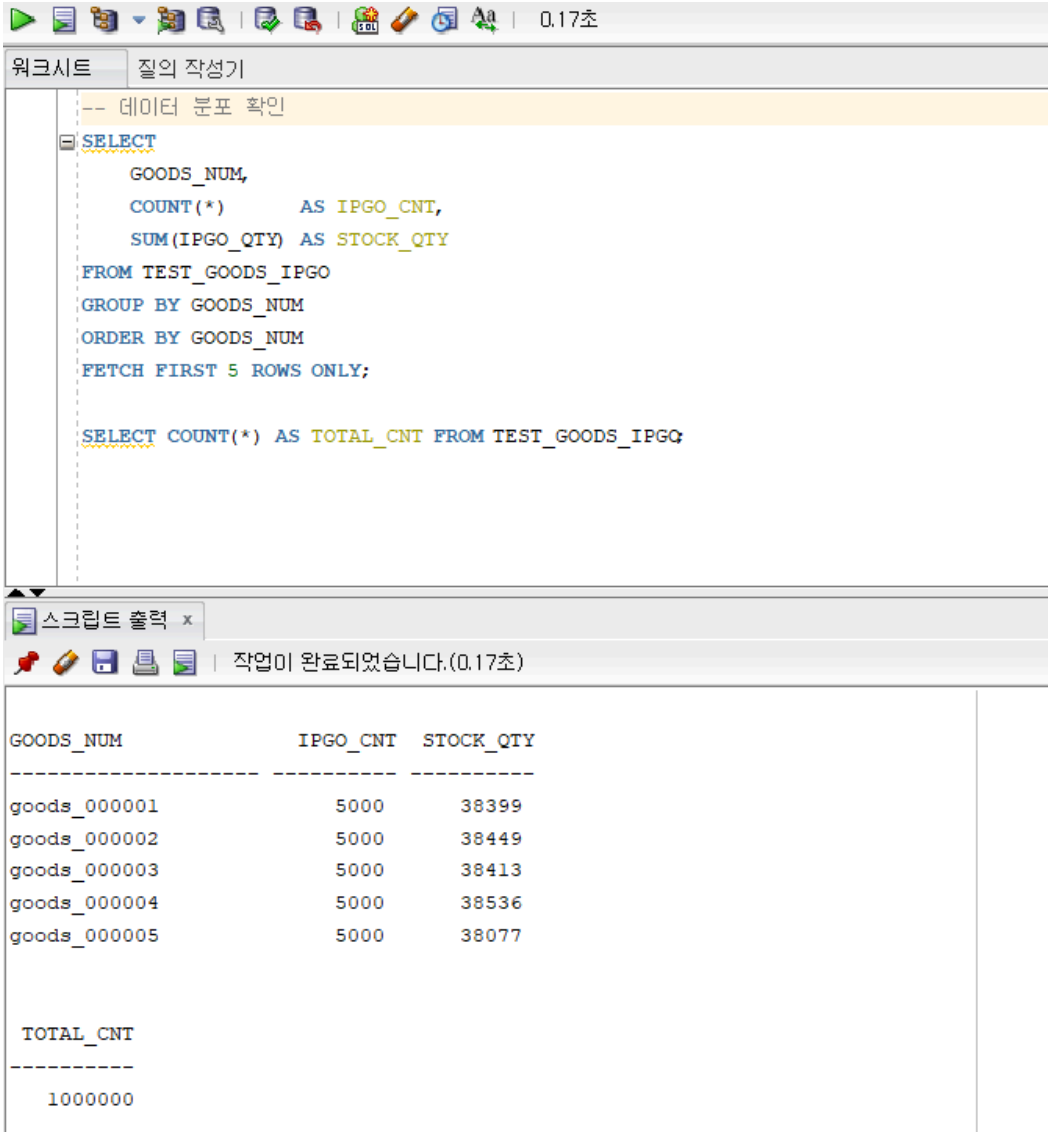
3. 테스트 데이터 분포

상품 200개에 걸쳐 균등 분산. 옵티마이저가 현실적인 판단을 내릴 수 있도록

각 상품당 5,000건의 수불부 이력을 삽입했다.

IPGO_QTY는 실제 수불부 패턴을 반영하여 입고(양수) / 출고·취소(음수)를 3:2:1 비율로 혼합했다.

> **데이터 규모 근거**: 수불부(GOODS_IPGO)는 입고/출고/취소 모든 재고 변동을 INSERT로 누적하는 테이블이다. 일평균 10건 변동 기준 약 1.5년 운영 시 상품당 약 5,000건이 쌓인다. 200개 상품 × 5,000건 = 100만 건은 현실적인 운영 시나리오다.

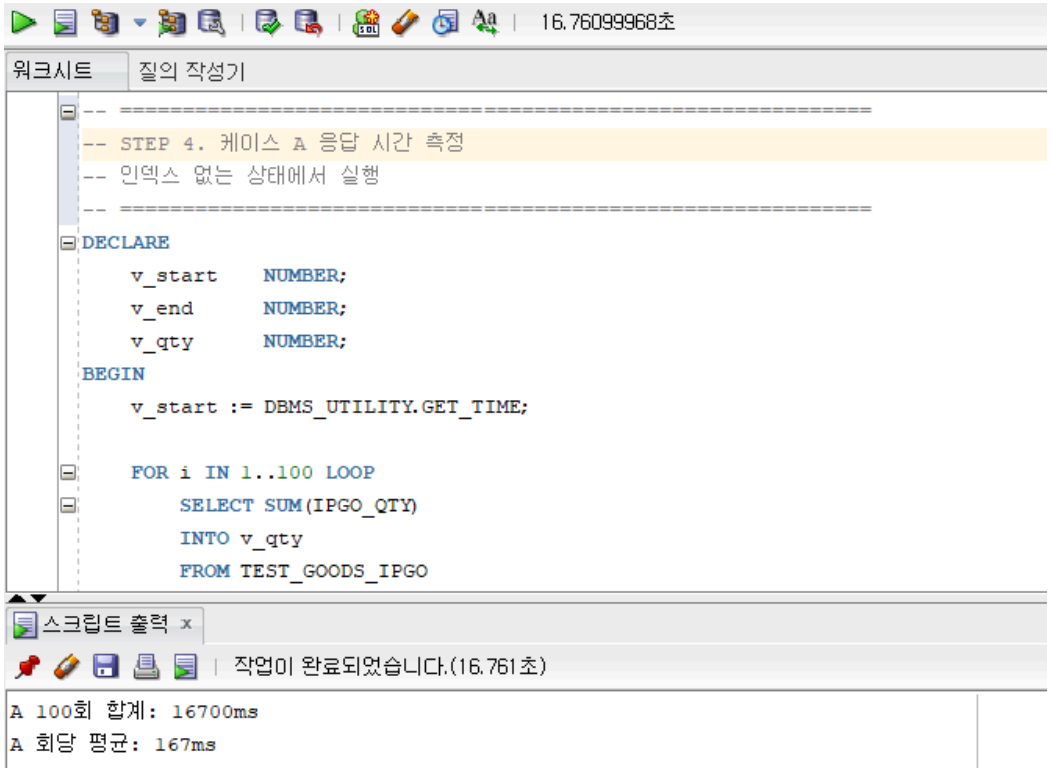


데이터 분포 확인: 상품별 5,000건 균등 분산, 총 100만 건

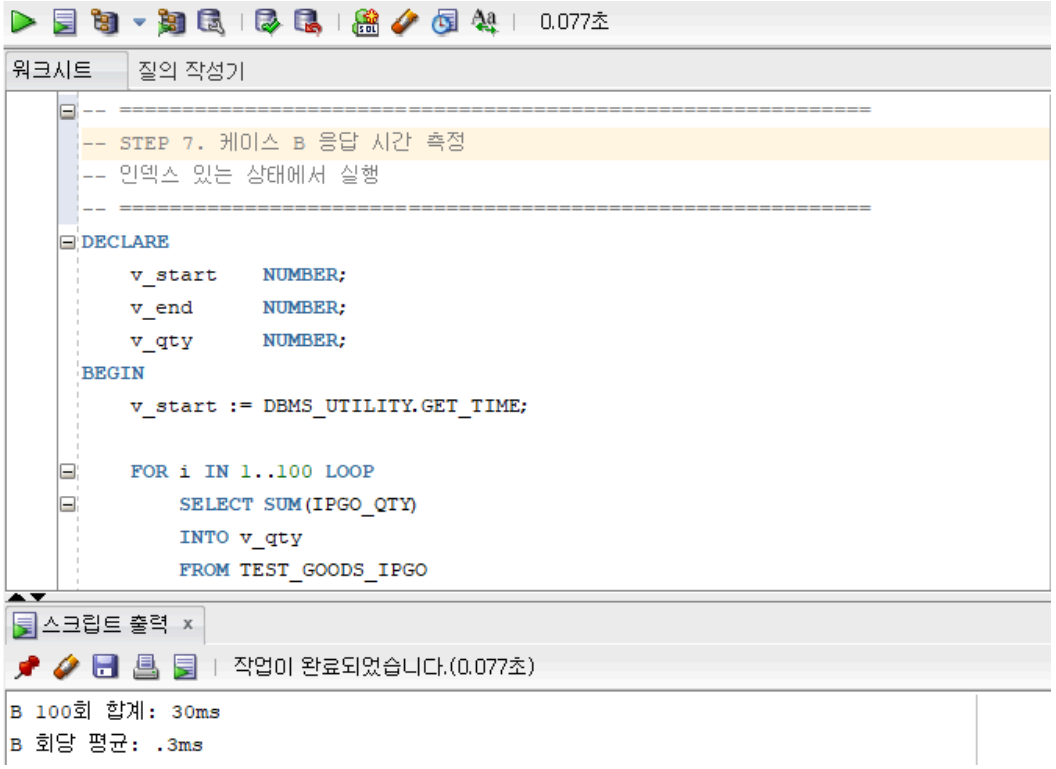
4. 측정 결과

4.1 응답 시간 비교

항목	인덱스 X	인덱스 O
100회 합계	16,700ms	30ms
회당 평균	**167ms**	**0.3ms**
향상	**—**	**약 557배 향상**



케이스 A 응답 시간 (100회 합계 16,700ms, 회당 평균 167ms)



케이스 B 응답 시간 (100회 합계 30ms, 회당 평균 0.3ms)

4.2 실행 계획 비교

항목	인덱스 X	인덱스 O	개선율
Cost	1,654	22	98.7% 감소
Operation	TABLE ACCESS FULL	INDEX RANGE SCAN	—
테이블 접근	있음	없음	완전 제거
Predicate	filter()	access()	—

0.076초

워크시트 질의 작성기

```
-- ============================================================
-- STEP 3. 케이스 A 실행 계획 - 인덱스 없음
-- ============================================================
EXPLAIN PLAN FOR
    SELECT SUM(IPGO_QTY) AS STOCK_QTY
    FROM TEST_GOODS_IPGO
    WHERE GOODS_NUM = 'goods_000001';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(FORMAT => 'ALL'));
```

스크립트 출력

작업이 완료되었습니다.(0.076초)

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 2650187747

-------------------------------------------------------------------------------
| Id  | Operation          | Name            | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |                 |     1 |    17 |  1654   (2)| 00:00:01 |
|   1 |  SORT AGGREGATE    |                 |     1 |    17 |            |          |
|*  2 |   TABLE ACCESS FULL| TEST_GOODS_IPGO |  5000 | 85000 |  1654   (2)| 00:00:01 |
-------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - filter("GOODS_NUM"='goods_000001')
```

케이스 A 실행 계획 (TABLE ACCESS FULL, Cost 1654)

0.07초

워크시트 질의 작성기

```
-- ============================================================
-- STEP 6. 케이스 B 실행 계획 - 복합 인덱스 있음
-- ============================================================
EXPLAIN PLAN FOR
    SELECT SUM(IPGO_QTY) AS STOCK_QTY
    FROM TEST_GOODS_IPGO
    WHERE GOODS_NUM = 'goods_000001';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(FORMAT => 'ALL'));
```

스크립트 출력

작업이 완료되었습니다.(0.07초)

```
Plan hash value: 951726604

-----------------------------------------------------------------------------------------
| Id  | Operation         | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |                       |     1 |    17 |    22   (0)| 00:00:01 |
|   1 |  SORT AGGREGATE   |                       |     1 |    17 |            |          |
|*  2 |   INDEX RANGE SCAN| IDX_TEST_IPGO_NUM_QTY |  5000 | 85000 |    22   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("GOODS_NUM"='goods_000001')
```

케이스 B 실행 계획 (INDEX RANGE SCAN, Cost 22)

---

5. 문제 분석

5.1 인덱스 없을 때

```
1. TEST_GOODS_IPGO 테이블 전체 스캔 (100만 건)
2. 각 행의 GOODS_NUM 확인 (filter 조건)
3. 일치하면 IPGO_QTY 집계

→ 100만 건 모두 읽음 (Cost 1,654)
```

5.2 복합 인덱스 적용 후

```
1. 인덱스에서 GOODS_NUM = 'goods_000001' 탐색 (access 조건)
2. 해당 인덱스 엔트리에서 IPGO_QTY 바로 읽음
3. 집계

→ 테이블 접근 없이 인덱스만으로 처리 (Covering Index, Cost 22)
```

5.3 filter() vs access() 차이

filter()는 인덱스로 행을 찾은 뒤 조건을 다시 검사하는 방식이고, access()는 인덱스 탐색 자체가 조건 적용을 의미한다. TABLE ACCESS 단계가 사라진 것과 함께, filter → access 전환이 Covering Index 효과의 직접적인 증거다.

---

6. 성능 vs 트레이드오프

6.1 조회 성능

```
평균 응답 시간: 167ms → 0.3ms (약 557배 향상)
옵티마이저 Cost: 1,654 → 22 (98.7% 감소)
```

데이터 증가 시에도 INDEX RANGE SCAN은 GOODS_NUM 범위로 탐색이 한정되어 안정적인 성능 유지.

6.2 INSERT 트레이드오프

GOODS_IPGO는 재고 변동(입고/출고/취소)마다 INSERT가 발생하는 수불부 테이블이다. 복합 인덱스 적용 시 INSERT마다 PK 인덱스와 복합 인덱스 2개를 동시에 갱신해야 하므로 쓰기 비용이 증가한다.

단건 INSERT는 측정 도구 정밀도(10ms) 한계로 인덱스 유무 간 차이가 측정되지 않았으나, 대량 INSERT 환경에서는 인덱스 수에 비례한 오버헤드가 누적된다.

0.061초

워크시트 질의 작성기

```oracle_plsql
-- ==============================================================
-- STEP 8. INSERT 트레이드오프 측정 (1,000건 배치)
-- ==============================================================

-- 8-1. 인덱스 없을 때 INSERT 1,000건
DECLARE
    v_start    NUMBER;
    v_end      NUMBER;
    v_total_ms NUMBER := 0;
BEGIN
    FOR i IN 1..1000 LOOP
        v_start := DBMS_UTILITY.GET_TIME;

        INSERT INTO TEST_GOODS_IPGO VALUES (
            2000000 + i,
```

스크립트 출력

작업이 완료되었습니다.(0.061초)

```
INSERT 인덱스X 1000건 합계: 30ms
INSERT 인덱스X 건당 평균: .03ms
```

0.479초

워크시트 질의 작성기

```oracle_plsql
-- 8-2. 인덱스 있을 때 INSERT 1,000건
CREATE INDEX IDX_TEST_IPGO_NUM_QTY
    ON TEST_GOODS_IPGO (GOODS_NUM, IPGO_QTY);

DECLARE
    v_start    NUMBER;
    v_end      NUMBER;
    v_total_ms NUMBER := 0;
BEGIN
    FOR i IN 1..1000 LOOP
        v_start := DBMS_UTILITY.GET_TIME;

        INSERT INTO TEST_GOODS_IPGO VALUES (
            3000000 + i,
            'goods_000001',
```

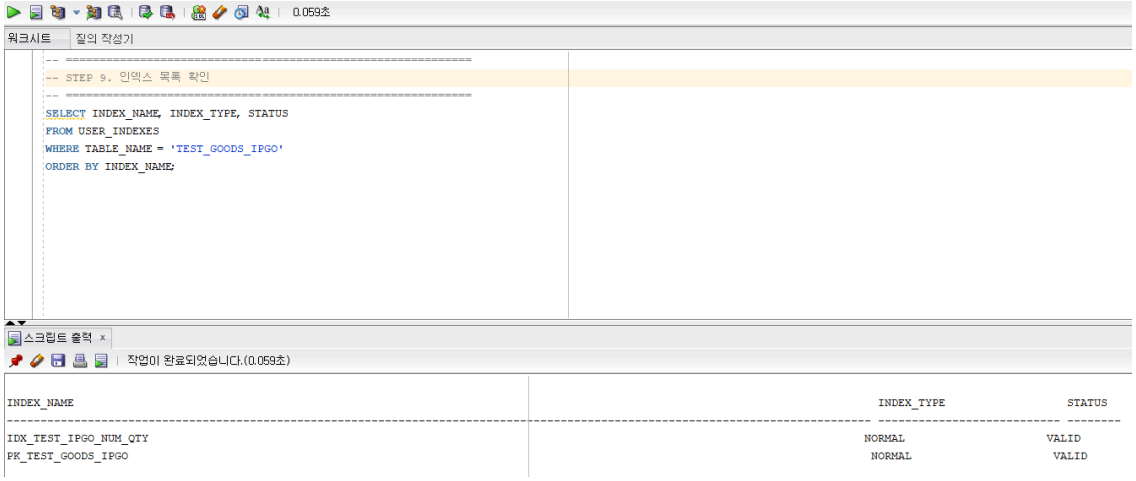
스크립트 출력

작업이 완료되었습니다.(0.479초)

```
Index IDX_TEST_IPGO_NUM_QTY이(가) 생성되었습니다.

INSERT 인덱스O 1000건 합계: 30ms
INSERT 인덱스O 건당 평균: .03ms
```

INSERT 트레이드오프: 인덱스X 0.03ms vs 인덱스O 0.03ms (단건 기준 측정 한계 내)



인덱스 목록: IDX_TEST_IPGO_NUM_QTY + PK_TEST_GOODS_IPGO 2개 유지 확인

관점	인덱스 없음	복합 인덱스 있음
SELECT (재고 집계)	느림 — Full Scan	빠름 — Index Only
INSERT (재고 변동 기록)	빠름 — 인덱스 1개 유지	상대적 부담 — 인덱스 2개 유지
데이터 증가에 따른 조회	선형 악화	안정적 유지

7. 결론

수불부 재고 집계 쿼리에 (GOODS_NUM, IPGO_QTY) Covering Index를 적용한 결과 Cost 98.7% 감소, 응답 시간 약 557배 향상을 확인했다.

TABLE ACCESS 단계 소멸과 Predicate의 filter → access 전환으로 Covering Index가 실제로 작동했음을 확인했다.

INSERT 시 추가 인덱스 유지 비용이 발생하나, 재고 집계 조회는 재고 변동 INSERT보다 빈번하게 호출되는 쿼리이므로 복합 인덱스 적용이 타당하다고 판단했다.